

# Modular MLOps Deployment Guide

The guide represents how the modular MLOps pipeline can be adopted and deployed in different cloud vendors. The modular components defined using IaC approach so switching cloud providers can be achieved easily without the need to rewriting the system.

## 1.1 Scope

- **AWS:** The pipeline is fully deployed and tested.
- **GCP:** Modular components provisioned successfully but orchestration is not ported.
- **Azure:** Modular components are not mapped. This guide includes expected equivalents based on the AWS/GCP mapping. However, these components are not provisioned or tested.

## 1.2 Provider-agnostic components

- **Application code:** The training script, validation gate logic, the deployment logic stays the same across different providers. However, orchestration of the pipeline such as calls to parameter storage (ssm), and instance controls (boto3) needs to be swapped with their equivalents using the provided mapping table.
- **Model registry (MLflow):** MLflow is an open-source and provider independent component that can be installed to and used in any cloud vendor.

| Component        | AWS(Terraform Resource)    | GCP (Terraform Resource)        | Azure (Terraform Source)       |
|------------------|----------------------------|---------------------------------|--------------------------------|
| Object Storage   | aws_s3_bucket              | google_storage_bucket           | azurerm_storage_container      |
| Trigger Function | aws_lambda_function        | google_cloudfunctions2_function | azurerm_linux_function_app     |
| Event Trigger    | aws_s3_bucket_notification | event_trigger block             | Event grid                     |
| Parameter Store  | aws_ssm_parameter          | google_secret_manager_secret    | Azurerm_key_vault_secret       |
| Compute          | aws_instance (ec2)         | google_compute_instance         | azurerm_linux_virtual_machine  |
| IAM Role         | aws_iam_role               | google_service_account          | Azurerm_user_assigned_identity |
| Provider         | hashicorp/aws              | hashicorp/google                | hashicorp/azurerm              |
| Instance control | boto3                      | Google Cloud Client             | Azure SDK for python           |

- **Deployment:** Deployment is served using FastAPI and can be used in any cloud vendor without any changes.

- **Overall workflow:** The flow of the workflow stays the same even though the cloud vendor gets swapped. The pipeline starts with the file upload to storage. An event trigger triggers the function to start the training instance. The model gets trained and starts the validation gate. Validation gate compares the model with the current champion, and if the current model outperforms the champion, it gets registered and deployed.

### 1.3 Components that require swapping

Each cloud component includes equivalent cloud functions that can easily be mapped by resource substitution and argument rename and addition. Table below represents the components equivalents in each cloud provider.

## 1.4 Prerequisites

- 1- Terraform installed.
- 2- A cloud account for the target provider is configured.
- 3- Repository installed
- 3- The dataset and training and deployment scripts are uploaded to the storage bucket.

## 1.5 Deployment

### 1.5.1 AWS

- 1- Configure the AWS provider in Terraform
- 2- Set AWS credentials
- 3- Make sure the user has right permission
- 4- Make sure necessary API's are enabled for the project
- 5- Run terraform init and terraform apply to provision the components
- 6- Add the dataset and training and deployment to dataset/ and scripts/ folders

### 1.5.2 GCP

- 1- Configure the GCP provider in Terraform
- 2- Authenticate using GCP credentials
- 3- Make sure the user has right permission
- 4- Make sure necessary API's are enabled for the project
- 5- Add a second google\_compute\_instance to serve monitoring, registry and deployment
- 6- Add the train.py and deployment.py files as metadata-startup-script argument to google compute instances .
- 7- Implement orchestration for GCP by switching boto3 packages with Google Cloud Client and aws\_ssm\_parameter with google\_secret\_manager\_secret
- 8- Run terraform init and terraform apply to provision the components
- 9- Add the dataset and training and deployment to dataset/ and scripts/ folders

### 1.5.3 Azure

- 1- Configure the Azure provider in Terraform
- 2- Authenticate using Azure credentials
- 3- Make sure the user has right permission
- 4- Make sure necessary API's are enabled for the project
- 5- Switch the defined components with their Azure equivalents using the table above.
- 6- Add the train.py and deployment.py files as custom\_data argument to azurerm\_linux\_virtual\_machine.

7- Implement orchestration for Azure by switching boto3 packages with Azure SDK for python and aws\_ssm\_parameter with azurerm\_key\_vault\_secret

8- Run terraform init and terraform apply to provision the components

9- Add the dataset and training and deployment to dataset/ and scripts/ folders

## 1.6 Logging

The application logs via Python's standard logging, the way the logs get transferred to monitoring tool Loki differs;

- **Trigger function logs:** Logs from the trigger function directly pushed to the Loki tool and can be seen under data section.

- **Compute logs:** Logs from the compute instance pushed to Loki via Promtail agent.

Logging in the pipeline is implemented open-source cloud-agnostic tools Grafana, Loki and Promtail.

These tools can be installed to any compute instance. Promtail is responsible for collecting logs, push it to Loki, and Grafana is used to visualise the logs.

## 1.7 Orchestration

Aws cloud components mapped to their equivalent components using terraform. However, full pipeline implementation including orchestration in GCP and Azure was out scope for this project.

- **AWS:** The pipeline is orchestrated.

- **GCP:** Orchestration can be implemented through switching boto3 packages with Google Cloud Client and aws\_ssm\_parameter with google\_secret\_manager\_secret

- **Azure:** Orchestration can be implemented through switching boto3 packages with Azure SDK for python and aws\_ssm\_parameter with azurerm\_key\_vault\_secret